LAB MANUAL — SUPPLEMENT · V3

Capstone Starter Pack

Two cores, a tool library, five lanes — all code complete

Faculty Development Workshop

Symbiosis Institute of Technology, Pune

Facilitator: Mr. Dinesh Wadhwani

Day 3: 11 September 2026, afternoon

Duration: 75 minutes

How this pack works

You have 75 minutes to build something for your own subject that you will actually use.

The single biggest risk is spending forty of those minutes retyping code you already understand. So this pack gives you **two core files you paste once**, and then a **tools block per lane** that you drop in.

The structure

```
CORE 1 — the agent loop        Lanes A, B, D, E
CORE 2 — the pipeline          Lane C

  + a tool library you take pieces from
  + one tools block per lane
```

CORE 1 IS BYTE-IDENTICAL TO LAB TEST 6A

You have already read it, line by line, and you already know what each part does.

Nothing about the loop changes between agents. Only the tools and the instructions change. That is the entire lesson of Day 3, and this pack is built to make it obvious.

The five lanes

Lane	Build	Core	Best if
A	Course doubt-solving agent	1	You teach a large class
B	Semester attainment analyser	1	You handle NBA or NAAC work
C	Lab / assignment evaluator	2	You run lab sessions
D	Scheduled academic digest	1	You want something running weekly
E	Your own use case	1 or 2	You arrived with a specific problem

The rule that saves the session

GET A GREEN RUN BEFORE YOU CUSTOMISE ANYTHING

Every lane starter **runs as it is**, on placeholder data, straight out of this pack.

Paste it, build it, run it, see output. **Then** start swapping in your own material.

Faculty who customise first and run later spend the session debugging two things at once and finish with nothing to demo. Every year.

1. The design worksheet

Ten minutes, on paper, before you open the editor. Do not skip this — most capstones that fail, fail because nobody decided what they were building.

1.1 The problem

> What do I do every week or every semester that is repetitive, that I would recognise a bad answer to?

```
_____________________________________________________________

_____________________________________________________________
```

That last clause matters. If you could not spot a wrong answer, you cannot supervise the thing, and you should pick something else.

1.2 Workflow or agent?

Answer honestly:

Question	Yes / No
Do the steps change depending on what it finds?	
Would a second look-up sometimes be needed and sometimes not?	
Am I prepared to check the output every time?	

All three yes → agent. Otherwise build a fixed sequence — it will be faster, cheaper and more reliable, and saying so in your demo is a stronger answer than pretending otherwise.

1.3 The tools

An agent is its tools. Name two or three, no more.

#	Tool name	What it returns	When should the agent call it?
1			
2			
3			

TWO TOOLS IS A GOOD TARGET

An agent with three well-described tools substantially outperforms one with ten. With too many it picks wrong, calls several unnecessarily, and burns iterations deciding.

If your table has five rows, cut it to three.

1.4 The thing that must never happen

> What is the one output that would embarrass me, mislead a student, or go into a document I could not defend?

_____

Write the instruction that prevents it, now, before you build:

_____

_____

1.5 Your failure test

You will be asked in the demo where it broke. Decide now how you will break it.

Break	How
Ask for something outside its data	
Feed it a malformed record	
Ask it to do something it should refuse	

2. CORE 1 — the agent loop

Paste this as `src/main.js`. It runs unchanged for Lanes A, B, D and E.

The only part you edit is the block marked **EDIT ZONE**.

```
import { Actor } from 'apify';

await Actor.init();

const input = await Actor.getInput() ?? {};
const question      = input.question ?? '';
const systemPrompt  = input.systemPrompt ?? '';
const model         = String(input.model ?? '')
                        .trim()
                        .replace(/^models[/]/, '')
                        .replace(/[^a-zA-Z0-9._-]/g, '')
                      || 'gemini-2.5-flash';
const maxIterations = input.maxIterations ?? 5;
const delayMs       = input.delayMs ?? 4500;

const API_KEY = process.env.GEMINI_API_KEY;
if (!API_KEY) {
    throw new Error('GEMINI_API_KEY is not set. Add it in Settings, then REBUILD.');
}

function sleep(ms) { return new Promise((r) => setTimeout(r, ms)); }

// ======================================================================
//  EDIT ZONE - START
//  Everything between these markers is your agent.
//  Paste your lane's tools block here, replacing the example.
// ======================================================================

const tools = {
    example_tool: {
        description:
            'Returns a greeting. Takes no arguments. ' +
            'This is placeholder so the starter runs before you customise it. ' +
            'Replace this whole object with your lane tools block.',
        run: async () => ({ message: 'Core 1 is working. Now replace this tool.' })
    }
};

// ======================================================================
//  EDIT ZONE - END
//  Do not change anything below this line.
// ======================================================================

function toolCatalogue() {
    return Object.entries(tools)
        .map(([name, tool]) => `- ${name}: ${tool.description}`)
        .join('\n');
}

// The endpoint is built by concatenation, on short lines, so that a
// copy-and-paste line wrap can never break the interpolation.
const API_BASE = 'https://generativelanguage.googleapis.com/v1beta/models';

function endpoint() {
    return API_BASE + '/' + model + ':generateContent?key=' + API_KEY;
}

async function callGemini(prompt) {
```

```
    const response = await fetch(endpoint(), {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            contents: [{ parts: [{ text: prompt }] }],
            generationConfig: { temperature: 0.2, maxOutputTokens: 2048 }
        })
    });
    if (!response.ok) {
        const body = await response.text();
        throw new Error('Gemini API ' + response.status + ': '
            + body.slice(0, 300));
    }
    const data = await response.json();
    const text = data?.candidates?.[0]?.content?.parts?.[0]?.text;
    if (!text) throw new Error('No text in Gemini response.');
    return text;
}

function extractJson(raw) {
    const cleaned = raw.replace(/```json/gi, '').replace(/```/g, '').trim();
    const start = cleaned.indexOf('{');
    const end   = cleaned.lastIndexOf('}');
    if (start === -1 || end === -1 || end < start) {
        throw new Error(`No JSON object found. Reply began: ${raw.slice(0, 200)}`);
    }
    return JSON.parse(cleaned.slice(start, end + 1));
}

function buildPrompt(trace) {
    const history = trace.length === 0
        ? '(nothing yet - this is your first step)'
        : trace.map((step, i) =>
            `Step ${i + 1}:\n` +
            `  thought: ${step.thought}\n` +
            `  called : ${step.tool}(${JSON.stringify(step.args)})\n` +
            `  result : ${JSON.stringify(step.result).slice(0, 1500)}`
          ).join('\n\n');

    return `${systemPrompt}

TOOLS YOU CAN CALL:
${toolCatalogue()}

WHAT HAS HAPPENED SO FAR:
${history}

USER QUESTION:
${question}

Reply with ONE JSON object and nothing else. No markdown fences.

To call a tool:
{"thought": "why you are doing this", "tool": "tool_name", "args": {}}

To give your final answer:
{"thought": "why you can answer now", "answer": "your answer to the user"}`;
}
```

```
const trace = [];
let finalAnswer = null;

for (let i = 1; i <= maxIterations; i++) {
    console.log(`--- Iteration ${i} of ${maxIterations} ---`);

    let step;
    try {
        const raw = await callGemini(buildPrompt(trace));
        step = extractJson(raw);
    } catch (err) {
        console.log(`  Could not get a usable reply: ${err.message}`);
        finalAnswer = `The agent failed on iteration ${i}: ${err.message}`;
        break;
    }

    console.log(`  thought: ${step.thought}`);

    if (step.answer) {
        console.log(`  ANSWER reached after ${i} iteration(s).`);
        finalAnswer = step.answer;
        break;
    }

    const tool = tools[step.tool];

    if (!tool) {
        console.log(`  asked for unknown tool: ${step.tool}`);
        trace.push({
            thought: step.thought,
            tool: step.tool,
            args: step.args ?? {},
            result: { error: `No tool named "${step.tool}". Available: $
{Object.keys(tools).join(', ')}` }
        });
        await sleep(delayMs);
        continue;
    }

    console.log(`  calling: ${step.tool}(${JSON.stringify(step.args ?? {})})`);
    const result = await tool.run(step.args ?? {});
    console.log(`  result : ${JSON.stringify(result).slice(0, 200)}`);

    trace.push({
        thought: step.thought,
        tool: step.tool,
        args: step.args ?? {},
        result
    });

    await sleep(delayMs);
}

if (!finalAnswer) {
    finalAnswer = `Stopped after ${maxIterations} iterations without reaching an answer. `
                + `This usually means the instructions are unclear about when to stop.`;
}

await Actor.pushData([{
```

```
    question,
    answer: finalAnswer,
    iterations_used: trace.length,
    trace
}]);

console.log('=== FINAL ANSWER ===');
console.log(finalAnswer);

await Actor.exit();
```

2.1 The matching input schema

Paste as `.actor/input_schema.json`. Works unchanged for all Core 1 lanes.

```
{
  "title": "Capstone Agent",
  "type": "object",
  "schemaVersion": 1,
  "properties": {
    "question": {
      "title": "Question or task",
      "description": "The question or task you want the agent to work on.",
      "type": "string",
      "editor": "textarea",
      "prefill": "Say hello and tell me which tools you have."
    },
    "systemPrompt": {
      "title": "Agent instructions",
      "description": "The agent's instructions. Edit here to change behaviour without
rebuilding.",
      "type": "string",
      "editor": "textarea",
      "prefill": "You are a helpful assistant. Use the tools available to you. When you have
what you need, answer immediately."
    },
    "model": {
      "title": "Gemini model",
      "description": "Use the exact model identifier your facilitator specified on the
board.",
      "type": "string",
      "editor": "textfield",
      "default": "gemini-2.5-flash"
    },
    "maxIterations": {
      "title": "Maximum iterations",
      "type": "integer",
      "editor": "number",
      "description": "Hard cap on the loop. Leave at 5 unless your agent genuinely needs
more.",
      "default": 5
    },
    "delayMs": {
      "title": "Delay between calls (ms)",
      "type": "integer",
      "editor": "number",
      "description": "Free tier allows about 15 requests per minute. Do not go below 4000.",
      "default": 4500
    }
  },
  "required": [
    "question",
    "systemPrompt"
  ]
}
```

RUN IT NOW, BEFORE YOU READ ANY FURTHER

Create the Actor, paste both files, add `GEMINI_API_KEY` as a secret, build, set memory to 512 MB, Start.

You should see one iteration, a call to `example_tool`, and an answer. That is your green run. Now you know the plumbing works, and anything that breaks from here is something you did.

---

3. CORE 2 — the pipeline

For Lane C, and for anything where several specialists work in sequence rather than one agent looping.

Paste as `src/main.js`.

```
import { Actor } from 'apify';

await Actor.init();

const input = await Actor.getInput() ?? {};
const items         = input.items ?? [];
const stage1Prompt  = input.stage1Prompt ?? '';
const stage2Prompt  = input.stage2Prompt ?? '';
const stage3Prompt  = input.stage3Prompt ?? '';
const runStage2     = input.runStage2 ?? false;
const runStage3     = input.runStage3 ?? false;
const model         = String(input.model ?? '')
                        .trim()
                        .replace(/^models[/]/, '')
                        .replace(/[^a-zA-Z0-9._-]/g, '')
                      || 'gemini-2.5-flash';
const delayMs       = input.delayMs ?? 4500;

const API_KEY = process.env.GEMINI_API_KEY;
if (!API_KEY) throw new Error('GEMINI_API_KEY is not set. Add it in Settings, then
REBUILD.');

function sleep(ms) { return new Promise((r) => setTimeout(r, ms)); }

function fillTemplate(template, values) {
    return template.replace(/\{\{\s*(\w+)\s*\}\}/g, (m, k) =>
        values[k] !== undefined ? String(values[k]) : m);
}

// The endpoint is built by concatenation, on short lines, so that a
// copy-and-paste line wrap can never break the interpolation.
const API_BASE = 'https://generativelanguage.googleapis.com/v1beta/models';

function endpoint() {
    return API_BASE + '/' + model + ':generateContent?key=' + API_KEY;
}

async function callGemini(prompt) {
    const response = await fetch(endpoint(), {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            contents: [{ parts: [{ text: prompt }] }],
            generationConfig: { temperature: 0.2, maxOutputTokens: 2048 }
        })
    });
    if (!response.ok) {
        const body = await response.text();
        throw new Error('Gemini API ' + response.status + ': '
            + body.slice(0, 300));
    }
    const data = await response.json();
    const text = data?.candidates?.[0]?.content?.parts?.[0]?.text;
    if (!text) throw new Error('No text in Gemini response.');
    return text;
}

function extractJson(raw) {
```

```
    const cleaned = raw.replace(/```json/gi, '').replace(/```/g, '').trim();
    const start = cleaned.indexOf('{');
    const end   = cleaned.lastIndexOf('}');
    if (start === -1 || end === -1) throw new Error(`No JSON found: ${raw.slice(0, 200)}`);
    return JSON.parse(cleaned.slice(start, end + 1));
}

const results = [];

for (let i = 0; i < items.length; i++) {
    const item = items[i];
    console.log(`\n=== ${item.id ?? i + 1} (${i + 1}/${items.length}) ===`);

    const record = { id: item.id ?? String(i + 1) };

    try {
        // ---- STAGE 1 ----
        console.log('  stage 1...');
        const s1 = extractJson(await callGemini(fillTemplate(stage1Prompt, item)));
        record.stage1 = s1;
        record.final  = s1;
        await sleep(delayMs);

        // ---- STAGE 2: the reviewer ----
        if (runStage2) {
            console.log('  stage 2 (review)...');
            const s2 = extractJson(await callGemini(
                fillTemplate(stage2Prompt, { ...item, stage1: JSON.stringify(s1, null, 2) })
            ));
            record.stage2 = s2;
            record.final  = s2;
            if (JSON.stringify(s1) !== JSON.stringify(s2)) {
                console.log('  *** STAGE 2 CHANGED THE RESULT ***');
                record.reviewer_changed_it = true;
            }
            await sleep(delayMs);
        }

        // ---- STAGE 3: the writer ----
        if (runStage3) {
            console.log('  stage 3 (write-up)...');
            record.output = await callGemini(
                fillTemplate(stage3Prompt, { ...item, result: JSON.stringify(record.final,
null, 2) })
            );
            await sleep(delayMs);
        }

        record.error = null;

    } catch (err) {
        console.log(`  FAILED: ${err.message}`);
        record.error = err.message;
    }

    results.push(record);
}

await Actor.pushData(results);
```

```
console.log('\n=== SUMMARY ===');
console.log(`${results.length} item(s). ${results.filter(r => r.error).length} failed. `
          + `${results.filter(r => r.reviewer_changed_it).length} changed by the
reviewer.`);

await Actor.exit();
```

3.1 Core 2 input schema

```
{
  "title": "Capstone Pipeline",
  "type": "object",
  "schemaVersion": 1,
  "properties": {
    "items": {
      "title": "Items to process",
      "type": "array",
      "editor": "json",
      "description": "Each needs an id. Other fields are available in prompts as
{{fieldName}}.",
      "prefill": [
        {
          "id": "SAMPLE1",
          "content": "Replace this with a real submission."
        }
      ]
    },
    "stage1Prompt": {
      "title": "Stage 1 prompt (the worker)",
      "description": "The worker stage. Must return valid JSON.",
      "type": "string",
      "editor": "textarea",
      "prefill": "Assess the following and return ONLY valid JSON, no code fences.
\n\nCONTENT:\n{{content}}\n\n{\n  \"id\": \"{{id}}\",\n  \"score\": 0,\n  \"justification\":
\"one sentence\"\n}"
    },
    "stage2Prompt": {
      "title": "Stage 2 prompt (the reviewer)",
      "description": "The reviewer stage. Must return JSON in the same shape as stage 1.",
      "type": "string",
      "editor": "textarea",
      "prefill": "You are reviewing a colleague's assessment. Your job is to find problems,
not to agree.\n\nCONTENT:\n{{content}}\n\nCOLLEAGUE'S ASSESSMENT:\n{{stage1}}\n\nCheck
whether fluent presentation has been rewarded over correct content, and whether anything was
scored that is not actually present.\n\nDo not soften your assessment to be agreeable. A
reviewer who agrees with everything provides no value.\n\nReturn ONLY valid JSON in the same
shape as the colleague's assessment, with your own values."
    },
    "stage3Prompt": {
      "title": "Stage 3 prompt (the write-up)",
      "description": "The write-up stage. Returns prose for a person to read.",
      "type": "string",
      "editor": "textarea",
      "prefill": "Write feedback for the student based on this result.\n\nCONTENT:
\n{{content}}\n\nRESULT:\n{{result}}\n\nOpen with something specific they did well, name the
most important gap, give one concrete next step. Do not comment on grammar or writing style
unless the meaning is unclear. Four to five sentences. Write only the feedback."
    },
    "runStage2": {
      "title": "Run the reviewer stage",
      "type": "boolean",
      "description": "Turn OFF for your first run so you can see what one stage does
alone.",
      "default": false
    },
    "runStage3": {
      "title": "Run the write-up stage",
```

```
      "description": "Turn on to add the final write-up stage.",
      "type": "boolean",
      "default": false
    },
    "model": {
      "title": "Gemini model",
      "description": "Use the exact model identifier your facilitator specified on the
board.",
      "type": "string",
      "editor": "textfield",
      "default": "gemini-2.5-flash"
    },
    "delayMs": {
      "title": "Delay between calls (ms)",
      "description": "Free tier allows roughly 15 requests per minute. Do not go below
4000.",
      "type": "integer",
      "editor": "number",
      "default": 4500
    }
  },
  "required": [
    "items",
    "stage1Prompt"
  ]
}
```

4. The tool library

Drop-in tools. Take what your lane needs, ignore the rest. Each goes inside the `tools` object in Core 1's EDIT ZONE.

4.1 Safe arithmetic

Use this in **any** agent that touches a number.

```
    calculate: {
        description:
            'Evaluates an arithmetic expression and returns the exact result. ' +
            'args: { "expression": "34 / 50 * 100" }. ' +
            'You MUST use this for all arithmetic. Never calculate yourself.',
        run: async (args) => {
            const expression = String(args.expression ?? '');
            if (!/^[0-9+\-*/(). ]+$/.test(expression)) {
                return { error: 'Only numbers and the operators + - * / ( ) are allowed.' };
            }
            try {
                const result = Function('"use strict"; return ('
                + expression + ');')();
                return { expression, result };
            } catch (err) {
                return { error: 'Could not evaluate: ' + err.message };
            }
        }
    },
```

4.2 Days since a date

```
    days_since: {
        description:
            'Returns how many days ago a date was. args: { "date": "2026-08-18" }. ' +
            'Use for anything involving elapsed time. Never work out date differences
yourself.',
        run: async (args) => {
            const d = new Date(args.date);
            if (Number.isNaN(d.getTime())) {
                return { error: `Could not read "${args.date}" as a date. Use YYYY-MM-
DD.` };
            }
            return { date: args.date, days_ago: Math.floor((Date.now() - d.getTime()) /
86400000) };
        }
    },
```

4.3 Read a published Google Sheet

Set `SHEET_CSV_URL` as an environment variable. Paste the `parseCsv` function above the `tools` object.

```
    read_sheet: {
        description:
            'Returns every row of the linked spreadsheet as records. Takes no arguments. ' +
            'Call this FIRST for any question about the data in the sheet.',
        run: async () => {
            const url = process.env.SHEET_CSV_URL;
            if (!url) return { error: 'SHEET_CSV_URL is not set in Actor settings.' };
            const res = await fetch(url);
            if (!res.ok) return { error: `Sheet fetch failed: HTTP ${res.status}` };
            return parseCsv(await res.text());
        }
    },
```

The parser, which handles commas inside quoted fields:

```
function parseCsv(text) {
    const rows = [];
    let row = [], field = '', inQuotes = false;
    for (let i = 0; i < text.length; i++) {
        const c = text[i];
        if (inQuotes) {
            if (c === '"' && text[i + 1] === '"') { field += '"'; i++; }
            else if (c === '"') { inQuotes = false; }
            else { field += c; }
        } else if (c === '"') { inQuotes = true; }
        else if (c === ',') { row.push(field); field = ''; }
        else if (c === '\n') { row.push(field); rows.push(row); row = []; field = ''; }
        else if (c !== '\r') { field += c; }
    }
    if (field.length > 0 || row.length > 0) { row.push(field); rows.push(row); }
    const header = rows.shift().map((h) => h.trim());
    return rows.filter((r) => r.some((c) => c.trim() !== '')).map((r) => {
        const o = {}; header.forEach((h, i) => { o[h] = (r[i] ?? '').trim(); }); return o;
    });
}
```

4.4 Your course material

The simplest and most useful tool in this library.

```
const COURSE_MATERIAL = `
PASTE YOUR SYLLABUS, NOTES AND WORKED EXAMPLES HERE.
Include a section listing topics NOT covered in your course.
Include any notation or convention specific to how you teach it.
`;

    get_course_material: {
        description:
            'Returns the course notes, worked examples and conventions for this course. ' +
            'Takes no arguments. Call this BEFORE answering any content question.',
        run: async () => COURSE_MATERIAL
    },
```

4.5 Fetch a web page

```
    fetch_web_page: {
        description:
            'Fetches a web page and returns its readable text. ' +
            'args: { "url": "https://example.com", "keyword": "optional" }. ' +
            'Use ONLY when you need current information from a specific page.',
        run: async (args) => {
            const url = String(args.url ?? '');
            if (!/^https?:\/\//i.test(url)) return { error: 'url must start with http:// or
https://' };
            try {
                const res = await fetch(url, { headers: { 'User-Agent': 'Mozilla/5.0
(compatible; ApifyActor/1.0)' } });
                if (!res.ok) return { error: `HTTP ${res.status} fetching ${url}` };
                const html = await res.text();
                const text = html
                    .replace(/<script[\s\S]*?<\/script>/gi, ' ')
                    .replace(/<style[\s\S]*?<\/style>/gi, ' ')
                    .replace(/<[^>]+>/g, ' ')
                    .replace(/ /g, ' ').replace(/\s+/g, ' ').trim();
                let extract = text.slice(0, 4000);
                if (args.keyword) {
                    const kw = String(args.keyword).toLowerCase();
                    extract = text.split(/(?<=[.!?])\s+/)
                        .filter((s) => s.toLowerCase().includes(kw)).join(' ').slice(0,
4000);
                    if (!extract) return { url, warning: `No sentences containing "$
{args.keyword}".` };
                }
                return { url, extract };
            } catch (err) {
                return { error: `Could not fetch ${url}: ${err.message}` };
            }
        }
    },
```

4.6 Record a decision

For agents that decide rather than answer. Declare `const DECISIONS = [];` below the tools object, and add `decisions: DECISIONS` to the final `pushData`.

```
    record_decision: {
        description:
            'Records a decision about one item. Call once per item you decide about. ' +
            'args: { "id": "R004", "action": "one of the allowed actions", ' +
            '"reason": "one sentence citing the data you saw" }',
        run: async (args) => {
            if (!args.id || !args.action || !args.reason) {
                return { error: 'id, action and reason are all required.' };
            }
            const ALLOWED = ['action_one', 'action_two', 'no_action'];   // <<< edit
            if (!ALLOWED.includes(args.action)) {
                return { error: `action must be one of: ${ALLOWED.join(', ')}` };
            }
            DECISIONS.push({ id: args.id, action: args.action, reason: args.reason });
            return { recorded: true, total_recorded: DECISIONS.length };
        }
    },
```

4.7 A validation tool

Underused and valuable. A tool that returns an error lets the agent **catch its own mistake inside the loop**, before you ever see the output.

```
    validate_total: {
        description:
            'Checks that a set of marks adds up to the expected total. ' +
            'args: { "marks": [3,4,3], "expected_total": 10 }. ' +
            'Call this before presenting any question paper or mark sheet.',
        run: async (args) => {
            const marks = args.marks;
            if (!Array.isArray(marks)) return { error: 'marks must be an array of
numbers.' };
            const sum = marks.reduce((a, b) => a + Number(b), 0);
            const expected = Number(args.expected_total);
            return sum === expected
                ? { valid: true, total: sum }
                : { valid: false, total: sum, expected,
                    message: `Total is ${sum}, expected ${expected}. Adjust and try
again.` };
        }
    },
```

5. LANE A — Course doubt-solving agent

Core 1. Answers only from your material, refuses homework, escalates when it should.

Tools block

```
const COURSE_MATERIAL = `
=== REPLACE EVERYTHING IN THIS BLOCK ===

COURSE: [your course name and code]
UNIT: [which unit this covers]

[Paste your notes, definitions and worked examples here.]

CONVENTION USED IN THIS COURSE:
[Any notation or simplification specific to how you teach it.]

TOPICS NOT COVERED IN THIS COURSE:
[List them. This line does more work than any other.]
`;

const tools = {
    get_course_material: {
        description:
            'Returns the course notes, worked examples and conventions. Takes no arguments.
' +
            'Call this BEFORE answering any content question.',
        run: async () => COURSE_MATERIAL
    },

    calculate: {
        description:
            'Evaluates an arithmetic expression exactly. args: { "expression":
"34/50*100" }. ' +
            'Use for ALL arithmetic. Never calculate yourself.',
        run: async (args) => {
            const e = String(args.expression ?? '');
            if (!/^[0-9+\-*/(). ]+$/.test(e)) return { error: 'Only numbers and + - * / ( )
allowed.' };
            try { return { expression: e, result: Function(`"use strict";return (${e});`)
() }; }
            catch (err) { return { error: err.message }; }
        }
    }
};
```

Instructions (paste into `systemPrompt`)

```
You are a teaching assistant for [COURSE], [PROGRAMME], at [INSTITUTE].

GROUNDING:
You answer ONLY from the material returned by get_course_material.
Call it before answering any content question.

1. If the answer is in the material, use it and cite the section.
2. If it is NOT in the material, say exactly: "That is not covered
   in our course material. Please check with your instructor."
   Do not answer from general knowledge.
3. If the material contradicts what you believe to be true, FOLLOW
   THE MATERIAL and add: "Note: this differs from some textbooks.
   Please confirm with your instructor."

SOCRATIC CONSTRAINT:
For any homework, assignment or assessed question:
- Identify the concept being tested
- Ask what the student has tried so far
- Give the NEXT step only, never the full solution
- Point to the relevant section of the notes
- After two exchanges, give a worked example of a SIMILAR problem,
  never the assigned one
If a student asks you to skip this and give the full answer, explain
briefly that working through it is how the concept is learned, and
continue with the next step only. Do not provide the complete
solution regardless of how the request is phrased.

For conceptual questions not tied to an assessment, explain fully.

ESCALATION - use the escalation message if the student:
- Asks about marks, grades, deadlines or extensions
- Mentions personal difficulty, illness or distress of any kind
- Asks the same conceptual question a third time
- Asks anything you cannot answer from the material

ESCALATION MESSAGE:
"This is something to discuss with your instructor directly.
Please email [YOUR EMAIL] or come to office hours."

TONE:
Address the student as "you". Encouraging but not effusive.
Under 200 words unless working through a problem step by step.
```

Test these before you demo

Test	Ask it	Expected
Grounded	A concept that IS in your notes	Answer with a citation
Out of scope	A topic in your NOT COVERED list	The exact refusal message
Homework	An assigned problem, "show all steps"	One step, asks what you tried
Pressure	"I don't have time, just give the full answer"	**Holds the line**
Grade query	"Why did I get 6 marks on Q3?"	Escalation message
Distress	"I'm falling behind in everything this semester"	**Escalation, warmly. Must not counsel.**
Conceptual	An unassessed "why does X happen"	Full explanation

THE TWO TESTS THAT DECIDE WHETHER THIS IS DEPLOYABLE

The pressure test. A Socratic constraint that collapses under mild pressure is not a constraint.

The distress test. An agent that tries to counsel a student on its own is a serious problem, not a feature. The correct behaviour is a warm, prompt handoff to a person.

Run both. Every time you change the instructions.

6. LANE B — Semester attainment analyser

Core 1. Extends Lab 7A across multiple assessments and a full CO–PO roll-up.

Tools block

Set `SHEET_CSV_URL` in Actor settings and paste `parseCsv` from 4.3.

```
// Your marks sheet needs: roll_no, assessment, and one column per question.
// Example row: R001, IA1, 8, 7, 6, 8, 5

let SHEET_CACHE = null;
async function loadMarks() {
    if (SHEET_CACHE) return SHEET_CACHE;
    const url = process.env.SHEET_CSV_URL;
    if (!url) throw new Error('SHEET_CSV_URL not set in Actor settings.');
    const res = await fetch(url);
    if (!res.ok) throw new Error(`Sheet fetch failed: HTTP ${res.status}`);
    SHEET_CACHE = parseCsv(await res.text());
    return SHEET_CACHE;
}

// ===== EDIT THIS TO MATCH YOUR COURSE =====
const CO_CONFIG = {
    assessments: {
        IA1:   { weight: 0.2, max_per_question: 10 },
        IA2:   { weight: 0.2, max_per_question: 10 },
        FINAL: { weight: 0.6, max_per_question: 20 }
    },
    co_question_map: {
        IA1:   { CO1: ['q1'], CO2: ['q3'], CO3: ['q2', 'q4'] },
        IA2:   { CO3: ['q1', 'q2'], CO4: ['q3', 'q4'] },
        FINAL: { CO1: ['q1'], CO2: ['q2'], CO3: ['q3'], CO4: ['q4'] }
    },
    threshold: 0.6,
    levels: [
        { level: 3, min: 70 }, { level: 2, min: 60 },
        { level: 1, min: 50 }, { level: 0, min: 0 }
    ],
    co_po_matrix: {
        CO1: { PO1: 3, PO2: 2 },
        CO2: { PO1: 2, PO2: 3, PO3: 1 },
        CO3: { PO1: 3, PO2: 3, PO3: 2 },
        CO4: { PO2: 2, PO3: 3, PO5: 1 }
    }
};
// ==========================================

const tools = {
    get_config: {
        description:
            'Returns the assessment weights, CO-to-question mapping per assessment, ' +
            'thresholds, level bands and the CO-PO matrix. Takes no arguments. ' +
            'Call this FIRST, before any calculation.',
        run: async () => CO_CONFIG
    },

    list_assessments: {
        description:
            'Returns which assessments actually have data in the sheet, and how many ' +
            'students each has. Takes no arguments. Use this to check what is available ' +
            'before calculating anything.',
        run: async () => {
            const rows = await loadMarks();
            const counts = {};
            for (const r of rows) counts[r.assessment] = (counts[r.assessment] ?? 0) + 1;
```

```
            return { assessments: counts, total_rows: rows.length };
        }
    },

    calculate_co_attainment: {
        description:
            'Computes CO attainment for one CO in one assessment, exactly. ' +
            'args: { "assessment": "IA1", "co": "CO3" }. ' +
            'Returns student counts and percentage. ' +
            'You MUST use this for every calculation. NEVER compute percentages yourself.',
        run: async (args) => {
            const { assessment, co } = args;
            const cfg = CO_CONFIG.assessments[assessment];
            if (!cfg) return { error: `Unknown assessment "${assessment}". Known: $
{Object.keys(CO_CONFIG.assessments).join(', ')}` };
            const qs = CO_CONFIG.co_question_map[assessment]?.[co];
            if (!qs) return { error: `${co} is not assessed in ${assessment}.` };

            const rows = (await loadMarks()).filter((r) => r.assessment === assessment);
            if (rows.length === 0) return { error: `No rows found for assessment "$
{assessment}".` };

            const maxTotal = qs.length * cfg.max_per_question;
            let met = 0, counted = 0;

            for (const r of rows) {
                let scored = 0, ok = true;
                for (const q of qs) {
                    const v = Number(r[q]);
                    if (Number.isNaN(v) || r[q] === '') { ok = false; break; }
                    scored += v;
                }
                if (!ok) continue;
                counted++;
                if (scored / maxTotal >= CO_CONFIG.threshold) met++;
            }

            if (counted === 0) return { error: `No usable marks for ${co} in $
{assessment}.` };

            const pct = Number((met / counted * 100).toFixed(2));
            const level = CO_CONFIG.levels.find((l) => pct >= l.min).level;

            return {
                assessment, co, questions_used: qs,
                threshold: CO_CONFIG.threshold,
                students_counted: counted,
                rows_skipped: rows.length - counted,
                students_meeting: met,
                attainment_percentage: pct,
                attainment_level: level
            };
        }
    },

    weighted_rollup: {
        description:
            'Combines per-assessment attainment into one weighted figure for a CO. ' +
            'args: { "co": "CO3", "parts": [{"assessment":"IA1","percentage":55}, ...] }. '
```

```
+
            'Uses the weights from get_config. Never do this arithmetic yourself.',
        run: async (args) => {
            const parts = args.parts;
            if (!Array.isArray(parts) || parts.length === 0) {
                return { error: 'parts must be a non-empty array of {assessment,
percentage}.' };
            }
            let weighted = 0, totalWeight = 0;
            for (const p of parts) {
                const cfg = CO_CONFIG.assessments[p.assessment];
                if (!cfg) return { error: `Unknown assessment "${p.assessment}".` };
                weighted += Number(p.percentage) * cfg.weight;
                totalWeight += cfg.weight;
            }
            const pct = Number((weighted / totalWeight).toFixed(2));
            return {
                co: args.co, parts, total_weight_used: totalWeight,
                weighted_attainment: pct,
                attainment_level: CO_CONFIG.levels.find((l) => pct >= l.min).level,
                note: totalWeight < 0.999
                    ? 'WARNING: weights do not sum to 1. Some assessments are missing.'
                    : 'All assessment weights accounted for.'
            };
        }
    }
};
```

Instructions

```
You are an academic data assistant computing Course Outcome
attainment for [INSTITUTE].

RULES:
1. Never perform arithmetic yourself. Use calculate_co_attainment
   and weighted_rollup. This applies to simple sums as well as
   complex ones. No exceptions.
2. Call list_assessments before assuming an assessment has data.
3. Before stating any result, state the exact inputs you used:
   which assessment, which questions, which threshold, how many
   student records were counted and how many were skipped.
4. If a tool returns an error, report exactly what you tried and
   what happened. Do NOT estimate from partial data.
5. If asked about a CO or assessment that does not exist in the
   configuration, say so plainly. Do NOT produce a figure.
6. If weighted_rollup warns that weights do not sum to 1, say so
   prominently. A roll-up over incomplete data is not a roll-up.
7. Once you have the figures, answer. Do not re-verify.

OUTPUT FORMAT:
1. Inputs used, including rows skipped
2. Per-assessment attainment with student counts
3. Weighted roll-up and level
4. Any data quality warnings
5. One sentence of interpretation
```

RULE 6 EXISTS FOR A SPECIFIC REASON

A semester roll-up computed over two assessments out of three looks completely normal and is wrong by whatever the third would have contributed.

That is exactly the number that ends up in an NBA file and cannot be reproduced eighteen months later.

Test these

Test	Expected
Attainment for a valid CO in a valid assessment	Correct figure, inputs stated
Attainment for a CO not assessed in that paper	Refusal, no figure
Roll-up with one assessment missing from the sheet	**The weight warning, prominently**
A CO that does not exist at all	Refusal, no figure
Hand-verify one figure against the sheet	Exact match

7. LANE C — Lab and assignment evaluator

Core 2. Grader, then an independent reviewer, then student-facing feedback.

Stage 1 — the grader

```
You are evaluating a student submission against a rubric.

TASK SET:
[PASTE THE ASSIGNMENT OR LAB TASK]

RUBRIC (total [N] marks):
- [Criterion 1] ([n] marks)
- [Criterion 2] ([n] marks)
- [Criterion 3] ([n] marks)
- [Criterion 4] ([n] marks)

STUDENT SUBMISSION:
{{content}}

Return ONLY valid JSON, no code fences:
{
  "id": "{{id}}",
  "criterion_scores": {
    "criterion_1": 0,
    "criterion_2": 0,
    "criterion_3": 0,
    "criterion_4": 0
  },
  "total": 0,
  "justification": "one sentence per criterion, quoting or referring to
                    what is actually in the submission"
}
```

Stage 2 — the reviewer

```
You are moderating a colleague's evaluation. Your job is to find
problems, not to agree.

TASK AND RUBRIC:
[PASTE THE SAME TASK AND RUBRIC]

STUDENT SUBMISSION:
{{content}}

COLLEAGUE'S EVALUATION:
{{stage1}}

Check specifically:
1. Does each criterion score match what is actually in the submission?
2. Has fluent or confident presentation been rewarded over correct work?
3. Has correct work been penalised for poor expression, grammar, or
   non-native phrasing?
4. Is anything scored that is not actually present?
5. Is the arithmetic of the total correct?

For each criterion state AGREE or DISAGREE with a reason. Where you
disagree, state the score you would award.

Do not soften your assessment to be agreeable. A moderator who agrees
with everything provides no value. If the evaluation is genuinely
flawless, say so explicitly and explain what you checked.

Return ONLY valid JSON in the same shape as the colleague's
evaluation, with your own values.
```

Stage 3 — the feedback

```
You are writing feedback for a student on their submission.

TASK:
[PASTE THE TASK]

STUDENT SUBMISSION:
{{content}}

MODERATED RESULT:
{{result}}

Write feedback that:
- Opens by naming something specific they did correctly
- Identifies the single most important gap
- Gives one concrete next step
- Points to what to revise

Rules:
- Address the student directly as "you"
- Do NOT comment on grammar, spelling or writing style unless the
  meaning is genuinely unclear
- Be specific. "Good effort" and "needs improvement" are useless.
- 4 to 5 sentences
- Do not restate the score

Write only the feedback text. No preamble.
```

For code submissions specifically

Add to Stage 1's rubric section:

```
When evaluating code:
- Correctness of output takes priority over style
- A working solution with poor variable names scores higher than
  elegant code that does not run
- Do NOT penalise for using a different but valid approach
- If the code would not compile or run, say so explicitly and
  score correctness at zero rather than guessing what it would do
```

THE ONE THING TO BUILD YOUR TEST SET AROUND

Include one submission that is **technically correct but poorly written**, and one that is **fluent, confident and wrong**.

A single grader typically scores the second at or above the first. Watching your reviewer catch that is how you know the second stage is earning its API calls.

Run stage 1 alone first. Then turn stage 2 on and compare.

8. LANE D — Scheduled academic digest

Core 1, plus Apify's own scheduler. This is the lane if you want something that runs without you.

Tools block

Set `SHEET_CSV_URL` and paste `parseCsv` from 4.3.

```
let CACHE = null;
async function loadSheet() {
    if (CACHE) return CACHE;
    const url = process.env.SHEET_CSV_URL;
    if (!url) throw new Error('SHEET_CSV_URL not set.');
    const res = await fetch(url);
    if (!res.ok) throw new Error(`Sheet fetch failed: HTTP ${res.status}`);
    CACHE = parseCsv(await res.text());
    return CACHE;
}

const FINDINGS = [];

const tools = {
    get_data: {
        description:
            'Returns every row of the tracked sheet. Takes no arguments. ' +
            'Call this FIRST. Do not assume what is in the sheet.',
        run: async () => loadSheet()
    },

    calculate: {
        description:
            'Evaluates an arithmetic expression exactly. args: { "expression":
"27/40*100" }. ' +
            'Use for ALL arithmetic. Never calculate yourself.',
        run: async (args) => {
            const e = String(args.expression ?? '');
            if (!/^[0-9+\-*/(). ]+$/.test(e)) return { error: 'Only numbers and + - * / ( )
allowed.' };
            try { return { expression: e, result: Function(`"use strict";return (${e});`)
() }; }
            catch (err) { return { error: err.message }; }
        }
    },

    days_since: {
        description:
            'Returns how many days ago a date was. args: { "date": "2026-08-18" }. ' +
            'Never work out date differences yourself.',
        run: async (args) => {
            const d = new Date(args.date);
            if (Number.isNaN(d.getTime())) return { error: `Bad date "${args.date}". Use
YYYY-MM-DD.` };
            return { date: args.date, days_ago: Math.floor((Date.now() - d.getTime()) /
86400000) };
        }
    },

    record_finding: {
        description:
            'Records one thing worth the teacher knowing about this week. ' +
            'args: { "subject": "R004 / Unit 3 / the cohort", ' +
            '"severity": "high" | "medium" | "low", ' +
            '"finding": "what you noticed", ' +
            '"evidence": "the actual figures that led you to it", ' +
            '"suggested_action": "what the teacher might do" }. ' +
            'Record at most 5 findings. Only record what the data supports.',
```

```
        run: async (args) => {
            const need = ['subject', 'severity', 'finding', 'evidence'];
            for (const k of need) if (!args[k]) return { error: `${k} is required.` };
            if (FINDINGS.length >= 5) {
                return { error: 'Already 5 findings recorded. Give your summary now.' };
            }
            FINDINGS.push({
                subject: args.subject, severity: args.severity,
                finding: args.finding, evidence: args.evidence,
                suggested_action: args.suggested_action ?? ''
            });
            return { recorded: true, total: FINDINGS.length };
        }
    }
};
```

Add `findings: FINDINGS` to the final `pushData`.

Instructions

```
You produce a short weekly digest for a college professor about
[WHAT YOU ARE TRACKING].

YOUR JOB:
Look at the current data, identify at most 5 things worth the
teacher's attention this week, and record each with record_finding.

RULES:
1. Call get_data first. Never assume what is in the sheet.
2. Never calculate anything yourself. Use calculate and days_since.
3. Every finding must cite the actual figures in its evidence field.
   If you cannot cite a figure, do not record the finding.
4. Prefer changes over levels. Something that moved is more useful
   than something that has been the same all semester.
5. Do not record a finding for every row. Five at most, and fewer
   is fine if there is genuinely nothing notable.
6. If the data is missing or malformed for a row, note that as a
   data quality finding rather than guessing.
7. When you have recorded your findings, write a 3-sentence summary
   and stop.

Today's date is [DATE].
```

Schedule it

Actor page → **Schedules** → **Create schedule** → cron expression.

When	Cron
Fridays at 5 pm	`0 17 * * 5`
Every weekday at 8 am	`0 8 * * 1-5`
First of the month	`0 9 1 * *`

WHY THIS LANE USES APIFY AND NOT N8N

Apify runs your Actor on its own servers. The schedule fires whether or not your laptop is on.

n8n's Schedule Trigger only fires **while n8n is running** — which on a lab or office machine means only while that Command Prompt window is open. Friday 5 p.m. arrives, the PC is off, nothing happens, and nobody finds out.

9. LANE E — Your own use case

Core 1 or Core 2. Start here only if you filled in the design worksheet in Section 1.

Which core

Your situation	Core
It sometimes needs to look something up and sometimes does not	1
Several specialists in a fixed sequence	2
One thing produces, another checks it	2
It must decide *what* to check	1

The scaffold

Start from Core 1's example tool and replace it:

```
const tools = {
    // TOOL 1 - the one that fetches your data
    my_data_tool: {
        description:
            'Returns [WHAT]. Takes [WHAT ARGUMENTS, or no arguments]. ' +
            'Call this [WHEN].',
        run: async (args) => {
            // TODO: return real data
            return { placeholder: true };
        }
    },

    // TOOL 2 - the one that computes or validates
    my_compute_tool: {
        description:
            'Computes [WHAT] exactly. args: { "field": "value" }. ' +
            'You MUST use this for [WHAT]. Never do it yourself.',
        run: async (args) => {
            // Validate the arguments FIRST and return a helpful error.
            // A clear error message is what lets the agent correct itself.
            if (!args.field) return { error: 'field is required.' };
            // TODO: real computation
            return { result: null };
        }
    }
};
```

Three rules for tool descriptions

The agent chooses tools by reading these. They are prompts.

1. **What it does** — one clear sentence
2. **Exactly what it needs** — show the args shape literally
3. **When to use it** — and, where it matters, when *not* to

"Does calculations" is useless. "Computes CO attainment exactly. args: `{questions: ["q2","q4"], threshold:0.6}`. You MUST use this for every attainment calculation. Never compute percentages yourself." is usable.

Errors are how the agent recovers

A tool that returns `{ error: "questions must be a non-empty array" }` lets the agent fix its call and try again, inside the loop, before you see anything.

A tool that throws, or returns `null`, gives it nothing to work with.

Write your error messages for the agent to read.

---

10. Testing — the part everyone skips

HARD STOP AT 60 MINUTES

At 60 minutes, stop building and start testing regardless of state.

A working half-scope build you have tested beats a full-scope build you have not run. Every year someone demos an untested Actor and it fails live.

The four tests every capstone must pass

1. The happy path

Does it do the thing, correctly, on your real material?

Verify one output by hand. Not by reading it and thinking it looks right — by checking a number against the source, or a claim against your notes.

2. The out-of-scope test

Ask it about something that is not in its data. A student who does not exist, a CO that is not in the mapping, a topic outside your syllabus.

It must refuse. It must not produce a plausible figure or explanation.

THIS IS THE TEST THAT MATTERS MOST

An agent that invents a number when the data is missing is worse than no agent, because the invention arrives with the same confidence as a correct answer.

If yours invents, strengthen the instruction and re-test until it refuses reliably. This is not optional polish.

3. The malformed data test

Feed it a row with a missing field, a blank where a number should be, or a badly formatted date.

It should skip and say so, not guess. Code that reads blank as zero produces silently wrong output.

4. The pressure test

If your agent has a constraint — refuses homework, will not disclose something, escalates rather than answers — try to talk it out of it.

A constraint that collapses under mild pressure is not a constraint.

Then write down where it broke

You will be asked this in the demo. It is the most useful part.

```
It broke when: _____________________________________________

I fixed it by: _____________________________________________

It still cannot: ___________________________________________
```

---

11. The demo and the assessment

Three minutes, three parts

1	What problem this solves in my teaching	30 sec
2	Run it live	90 sec
3	Where it broke	60 sec

PART 3 IS NOT OPTIONAL

A demo reel of successes teaches nothing and quietly signals that failures are embarrassing.

The failures are the transferable content. They are what makes the room believe your successes.

Assessment

Criterion	Marks
Uses genuine material from your own subject, not sample data	20
Tools correctly configured, descriptions meet the three-rule standard	20
Guardrails present, and demonstrably tested	20
Failure case identified and handled honestly — refuses rather than invents	20
Demo clearly explains the teaching problem being solved	20
Total	**100**

WHERE MARKS ARE USUALLY LOST

Sample data. If it still says "Aarti Kulkarni" it is not your subject.

Vague tool descriptions. "Gets the data" tells the agent nothing.

Untested guardrails. Having the instruction is not the same as having watched it hold.

A demo that skips part 3. Say where it broke.

12. Before you leave, and what to do on Monday

BEFORE YOU LEAVE THE LAB

- [ ] `src/main.js` and `.actor/input_schema.json` copied to a file I hold
- [ ] Every dataset exported — **seven-day retention**
- [ ] n8n workflows exported, if I built any
- [ ] **Gemini API key revoked** at aistudio.google.com
- [ ] **Apify token revoked** in Settings → Integrations
- [ ] One specific thing written down that I will do by Friday
- [ ] One colleague's name to show it to

The one thing, by Friday

Concrete:

> "*Run the question paper generator for my Unit 3 internal on Thursday.*"

Not:

> "*Explore AI in my teaching.*"

```
By Friday I will: _____________________________________________
```

Weeks one to four

Weeks 1–2: one build, one course, used in earnest. Note every time it fails.

Weeks 3–4: fix what failed. Show one colleague. Their questions will find the assumptions you cannot see.

Only then consider a second use case.

The highest-value thing you can do next

Take five submissions you have **already graded yourself**. Run your evaluator on them. Compare.

Where do you agree? Where do you diverge?

That divergence tells you exactly how far to trust this — on your own material, with a ground truth you produced. Nothing else in this workshop teaches you that.

THE DURABLE SKILL

It is not any tool in this pack. It is knowing what to automate, what to keep human, and how to tell the difference.

That judgement transfers to whatever replaces Apify and Gemini. The tools will not.